

Interview Task: AI-Powered Media Processing Microservice

Objective

Build and deploy a microservice that accepts user-uploaded media files, processes them asynchronously through an AI pipeline, and returns enriched results. The system should reflect how real media processing pipelines and websites work where results are made available without blocking the user. Assume this is the backend infrastructure for a content platform where users upload images and the platform extracts structured metadata from them automatically.

Context & Assumptions

Users of this platform upload images (photographs, screenshots, scanned documents). Once uploaded, each file needs to be:

1. Stored durably
2. Queued for AI processing
3. Processed by a worker that calls an external AI endpoint
4. Enriched with the results and made queryable

The processing pipeline runs **three sequential AI tasks** on every image:

Step	Task	What it produces
1	Image Captioning	A natural language description of the image
2	Object/Label Detection	A list of objects, concepts or labels detected
3	Content Safety Check	A flag indicating whether the image is safe, and a category if not

You can use [Google Cloud Vision API](#) for label detection and content safety (SafeSearch), and [Hugging Face Inference API](#) (model: [Salesforce/blip-image-captioning-base](#)) for image captioning. Both have free tiers sufficient for this task. The point is not which model you pick- more about how you design the pipeline around it.

Requirements

Authentication

- Users must be able to sign up and log in.
- All endpoints below must be authenticated. Unauthenticated requests must be rejected.
- You may use any auth strategy (JWT, session-based, OAuth) but your choice must be documented and justified.

File Upload

- Accept image uploads in **JPG**, **PNG**, and **WEBP** formats only. Reject anything else with a clear error.
- Maximum file size: **5MB**. Enforce this at the API layer, not just the frontend.
- On upload, assign the file a unique job ID, store the file, create a job record in the database with status **pending**, and enqueue the job. Return the job ID to the user immediately. Do not make the user wait for processing.

Frontend

- A minimal UI where a user can:
 - Sign up / log in
 - Upload an image
 - See a list of their past jobs with statuses
 - Click into a job and see the full results (caption, labels, safety classification)
 - Retry a failed job from the UI
- The UI should reflect job status updates — either via polling or WebSockets. The approach is your choice, but must be documented.

Flagged Content

- If the content safety step returns any result other than **SAFE** (i.e. Google Vision's SafeSearch returns **LIKELY** or **VERY_LIKELY** for any category), mark the job's result as **flagged: true** and store the flagged category.
 - Flagged jobs must be surfaced distinctly in the user's job list.
 - Notify the user (in-app notification or email — your choice, document it) that their upload was flagged.
-

Tech Stack

Layer	Requirement
Application	MERN or PERN
Queue	Open-ended
Containerisation	Docker (mandatory). Kubernetes (bonus)
AI Endpoints	Google Cloud Vision API + Hugging Face Inference API (as specified above)
File Storage	Open-ended — local volume, S3, GCS, Cloudflare R2
Auth	Open-ended — document your choice
CI/CD	Open-ended — document your choice
Cloud Platform	Open-ended

A `docker-compose.yml` must be provided that spins up the full system locally: API service, worker service, queue, and database. The reviewer must be able to run `docker-compose up` and have a working system.

Deliverables

1. **GitHub repository** with the full source code
 2. **Deployed application URL** — the full system must be deployed and accessible, not just runnable locally
 3. **API collection** — Postman collection or OpenAPI/Swagger spec covering all endpoints
 4. **README.md** covering:
 - Architecture diagram (can be hand-drawn, Excalidraw, or any tool)
 - How to run locally
 - Environment variables and how to obtain API keys
 - Your assumptions and the decisions you made where the spec was open-ended
 - Known limitations or things you would do differently with more time
-

Evaluation Criteria

The requirements above are the baseline. What separates submissions is the quality of thinking behind the implementation.

- **Requirements coverage** — does the system do what is specified?
- **Architecture & design decisions** — Is the separation between API and worker well-reasoned? How are failures handled? How is state managed across services?
- **Code quality** — Is the code readable, consistently linted, and reasonably tested? Unit tests on the worker pipeline logic and retry behaviour are expected at minimum.
- **Documentation** — Are your choices explained? Could another engineer pick this up and extend it?
- **Bonus: Scalability** — How would this system behave under 10x load? Would adding more workers help? Are there bottlenecks? You don't need to solve these — articulate them.

Time Limit: 48 hours